AWS RDS MySQL Integration with Java Web Application

Project Overview

This project demonstrates the deployment of a **Java-based Login and User Registration web application** on **Amazon EC2**, integrated with **Amazon RDS MySQL** as the backend database.

The application code was deployed from the GitHub repository:

https://github.com/CharantejAkri/aws-rds-java.git

The main objective of this project is to validate end-to-end connectivity between:

- **EC2 (Application Server / MySQL Client)**
- **Amazon RDS MySQL (Database Server)**
- **Java JSP-based Login & Registration Application**

The project also verifies whether the user registration details are successfully stored in the database.

Amazon RDS provides a managed MySQL database instance that can be securely accessed from an application server running on EC2.

Architecture

```
User Browser
    |
    ▼
EC2 Instance (Tomcat + Java App)
    |
    ▼
Amazon RDS MySQL
Database: jwt
Table: users
```

AWS Services Used

- **Amazon EC2**
  - Hosts the Java web application
  - Acts as the application server and MySQL client
- **Amazon RDS (MySQL)**
  - Managed relational database service
  - Stores login and registration details
- **Security Groups**
  - Allows EC2 → RDS communication on port **3306**
  - Allows browser access on port **8080**
- **VPC**
  - Enables secure network communication between EC2 and RDS

Deployment Steps

1. Launch EC2 Instance

An Ubuntu EC2 instance was launched to host the application.

Installed Components

sudo apt update
sudo apt install openjdk-11-jdk -y
sudo apt install tomcat9 -y
sudo apt install mysql-client-core-8.0 -y

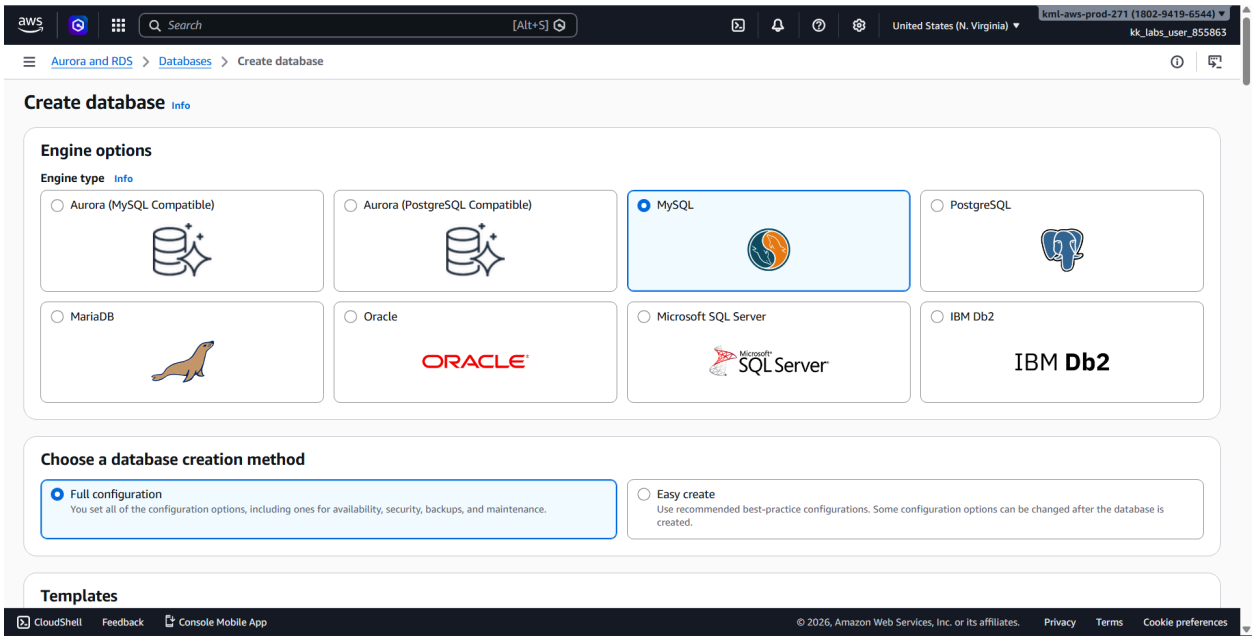
```
ubuntu@ip-172-31-26-67:~$ mysql
Command 'mysql' not found, but can be installed with:
sudo apt install mysql-client-core-8.0  # version 8.0.45-0ubuntu0.24.04.1, or
sudo apt install mariadb-client-core    # version 1:10.11.14-0ubuntu0.24.04.1
ubuntu@ip-172-31-26-67:~$ sudo apt install mysql-client-core-8.0 -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  mysql-client-core-8.0
0 upgraded, 1 newly installed, 0 to remove and 48 not upgraded.
Need to get 2740 kB of archives.
```

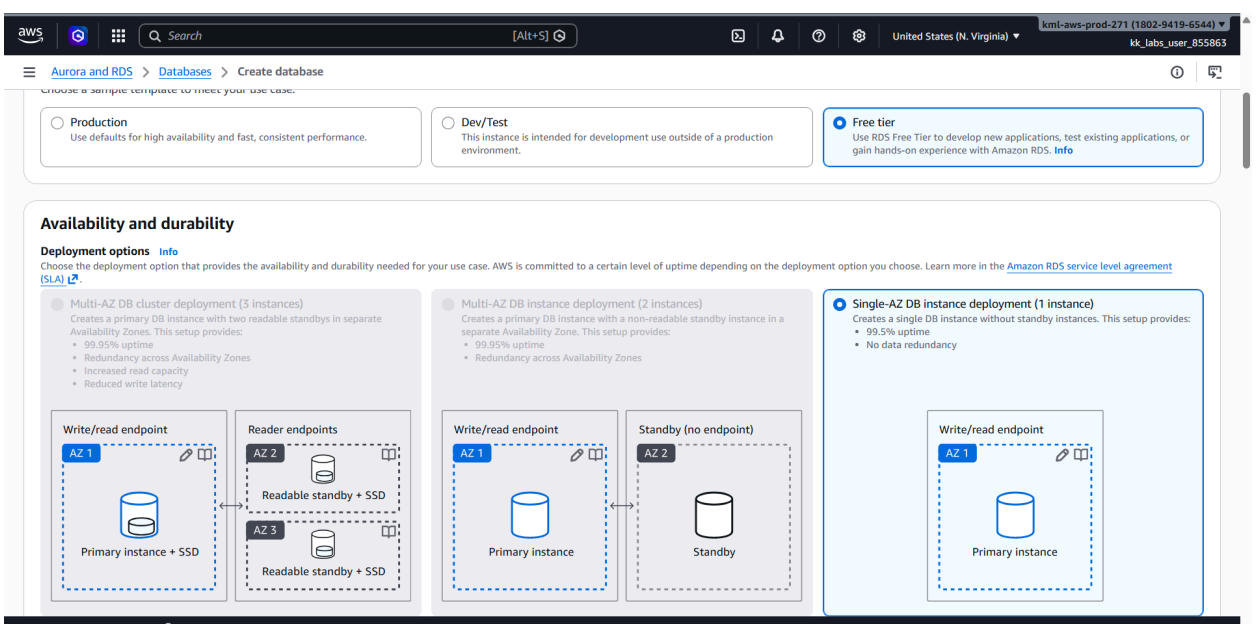
---

2. Create Amazon RDS MySQL Instance

A MySQL RDS instance was created with the following configuration:

Parameter	Value
Engine	MySQL
Port	3306
Database Name	jwt
Username	admin





aws | Search [Alt+S] | United States (N. Virginia) | kml-aws-prod-271 (1802-9419-6544) | kk_labs_user_855863

Aurora and RDS > Databases > Create database

Enable RDS Extended Support Info

Amazon RDS Extended Support is a paid offering. By selecting this option, you consent to being charged for this offering if you are running your database major version past the RDS end of standard support date for that version. Check the end of standard support date for your major version in the RDS for MySQL documentation.

DB instance identifier Info

Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.

DB-Task

The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

Credentials Settings

Master username Info

Type a login ID for the master user of your DB instance.

admin

1 to 16 alphanumeric characters. The first character must be a letter.

Credentials management

You can use AWS Secrets Manager or manage your master user credentials.

- Managed in AWS Secrets Manager - *most secure*
  RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.
- Self managed
  Create your own password or have RDS create a password that you manage.

Auto generate password

Amazon RDS can generate a password for you, or you can specify your own password.

Master password Info

••••••••

Minimum constraints: At least 8 printable ASCII characters. Can't contain any of the following symbols: / ' " @

aws | Search [Alt+S] | United States (N. Virginia) | kml-aws-

Aurora and RDS > Databases > Create database

Connectivity Info

Compute resource

Choose whether to set up a connection to a compute resource for this database. Setting up a connection will automatically change connectivity settings so that the compute resource can connect to this database.

- Don't connect to an EC2 compute resource
  Don't set up a connection to a compute resource for this database. You can manually set up a connection to a compute resource later.
- Connect to an EC2 compute resource
  Set up a connection to an EC2 compute resource for this database.

Virtual private cloud (VPC) Info

Choose the VPC. The VPC defines the virtual networking environment for this DB instance.

Default VPC (vpc-0fb9d17729e59232e)
6 Subnets, 6 Availability Zones

After a database is created, you can't change its VPC. Only VPCs with a corresponding DB subnet group are listed.

After a database is created, you can't change its VPC.

DB subnet group Info

Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB instance can use in the VPC that you selected.

default

Public access Info

- Yes
  RDS assigns a public IP address to the database. Amazon EC2 instances and other resources outside of the VPC can connect to your database. Resources inside the VPC can also connect to the database. Choose one or more VPC secu
  which resources can connect to the database.
- No
  RDS doesn't assign a public IP address to the database. Only Amazon EC2 instances and other resources inside the VPC can connect to your database. Choose one or more VPC security groups that specify which resources can connec

VPC security group (firewall) Info

Choose one or more VPC security groups to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.

User: arn:aws:iam::180294196544:user/kk_labs_user_855863 is not authorized to perform: rds:DescribeGlobalClusters on resource: arn:aws:rds::180294196544:global-cluster:* with an explicit deny in a service control policy: arn:aws:organizations::598274344262:policy/o-ptvbaba0eu/service_control_policy/p-nrgm8cly

Diagnose with Amazon Q

Databases (1)

Group resources | Modify | Actions | Create database

Filter by databases

1

DB identifier	Status	Role	Engine	Upgrade rollout order	Region ...	Size
db-task	Available	Instance	MySQL Co...	SECOND	us-east-1f	db.t4g.micro

3. Configure Security Groups

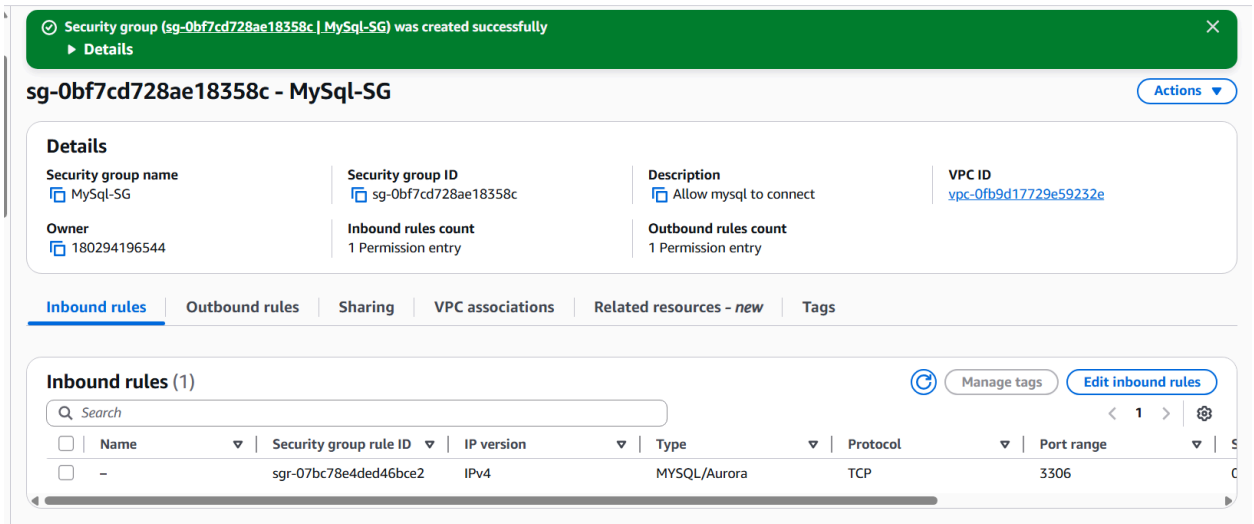
EC2 Security Group

Type	Port
SSH	22
HTTP	8080

RDS Security Group

Type	Port	Source
MySQL/Aurora	3306	EC2 Security Group

This configuration allows the application server to communicate securely with the RDS database.



---

4. Deploy Application from GitHub

The project code was cloned from the repository:

git clone https://github.com/CharantejAkri/aws-rds-java.git

Update login.jsp and userregistration.jsp in the src/main/webapp/

```
  GNU nano 7.2                                                    login.jsp
<%@ page import="java.sql.*"%>
<%
    String userName = request.getParameter("userName");
    String password = request.getParameter("password");
    Class.forName("com.mysql.jdbc.Driver");
    Connection con = DriverManager.getConnection("jdbc:mysql://db-task.cf0a28aaof6i.us-east-1.rds.amazonaws.com:3306/jwt","admin", "admin123");
    Statement st = con.createStatement();
    ResultSet rs;
    rs = st.executeQuery("select * from USER where userName='" +userName + "' and password='" +password + "'");
    if (rs.next()) {
        session.setAttribute("userName", userName);
        response.sendRedirect("success.jsp");
    } else {
        out.println("Invalid password <a href='index.jsp'>try again</a>");
    }
%>
```

```
  GNU nano 7.2                                          userRegistration.jsp *
<%@ page import="java.sql.*"%>
<%
    String userName = request.getParameter("userName");
    String password = request.getParameter("password");
    String firstName = request.getParameter("firstName");
    String lastName = request.getParameter("lastName");
    String email = request.getParameter("email");
    Class.forName("com.mysql.cj.jdbc.Driver");
    Connection con = DriverManager.getConnection("jdbc:mysql://db-task.cf0a28aaof6i.us-east-1.rds.amazonaws.com:3306/jwt", "admin", "admin123");
    Statement st = con.createStatement();
    int i = st.executeUpdate("insert into USER(first_name, last_name, email, username, password, regdate) values ('" + firstName + "','"
    + lastName + "','" + email + "','" + userName + "','" + password + "', CURDATE())");
    if (i > 0) {
        response.sendRedirect("welcome.jsp");

    } else {
        response.sendRedirect("index.jsp");
    }
%>
```

The application files were copied into Tomcat webapps:

sudo cp -r aws-rds-java/LoginWebApp /var/lib/tomcat9/webapps/

Restart Tomcat:

sudo systemctl restart tomcat9

Database Table Creation

The `users` table was created manually inside the `jwt` database.

```
USE jwt;

CREATE TABLE users (
   id INT AUTO_INCREMENT PRIMARY KEY,
   first_name VARCHAR(100),
   last_name VARCHAR(100),
   email VARCHAR(150),
   username VARCHAR(100),
   password VARCHAR(100),
   regdate DATE
);
```

Verification:

SHOW TABLES;

Output:

Users

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
4 rows in set (0.02 sec)

mysql> create database jwt;
Query OK, 1 row affected (0.04 sec)

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| jwt                |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.01 sec)

mysql> |
```

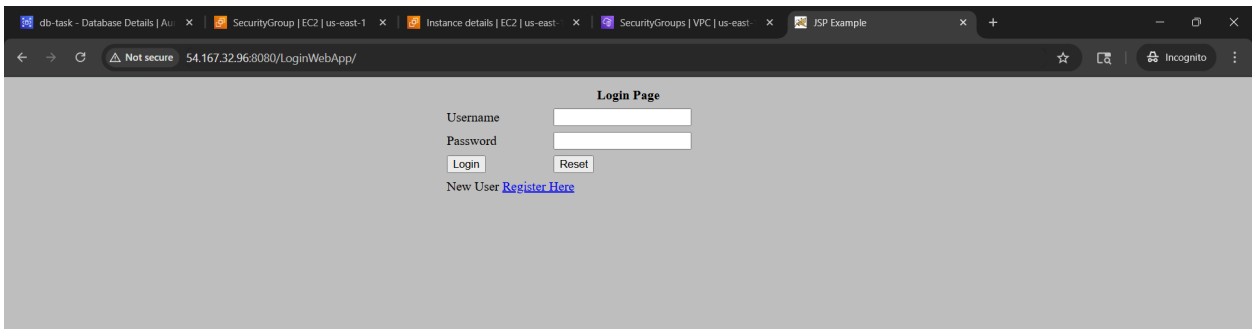
Application URLs

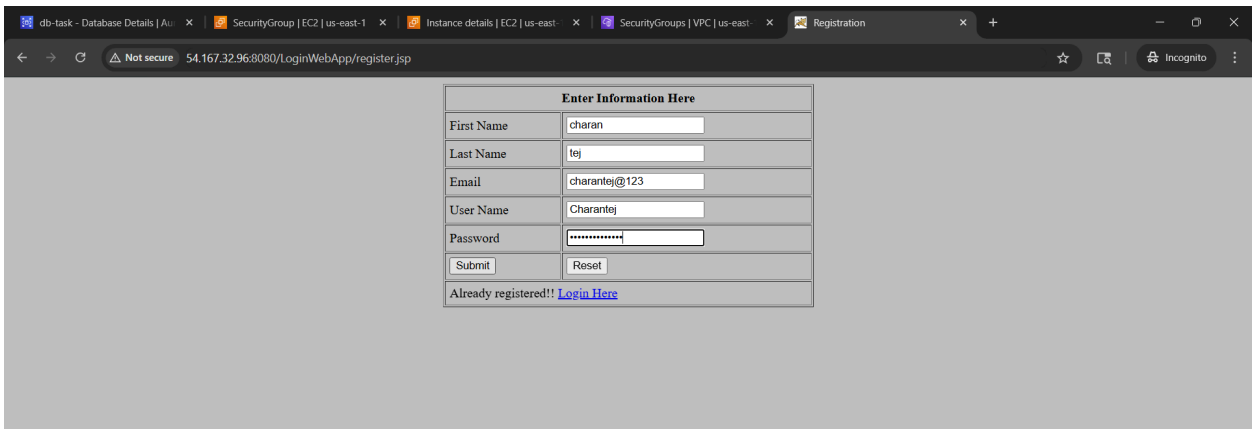
Registration Page

http://<EC2-Public-IP>:8080/LoginWebApp/userRegistration.jsp

Login Page

http://<EC2-Public-IP>:8080/LoginWebApp/login.jsp



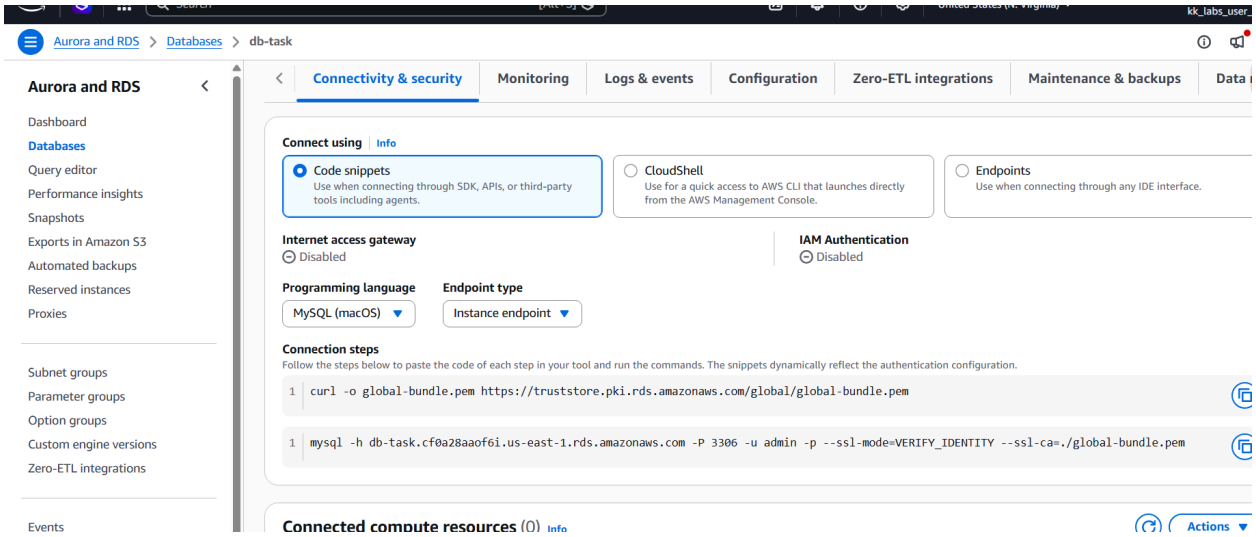


Database Verification

After submitting the registration form, the data was verified in Amazon RDS.

Login to MySQL client from EC2:

mysql -h <rds-endpoint> -u admin -p



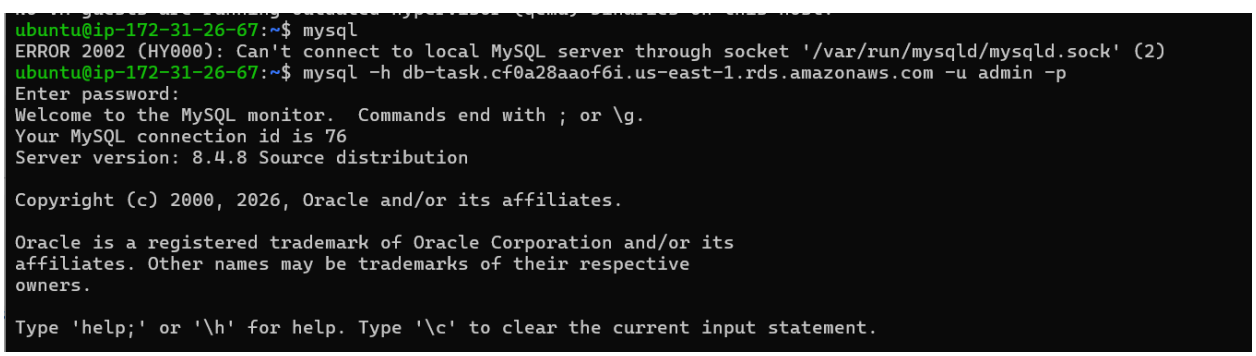
Run:

USE jwt;
SELECT * FROM users;

Example output:

```
+----+------------+-----------+------------------+----------+----------+------------+
| id | first_name | last_name | email            | username | password | regdate    |
+----+------------+-----------+------------------+----------+----------+------------+
|  1 | Charan     | Byna      | charan@email.com | charan   | ******   | 2026-04-16 |
+----+------------+-----------+------------------+----------+----------+------------+
```

This confirms that the user registration details are successfully stored in the database.



```
mysql> show tables;
+---------------+
| Tables_in_jwt |
+---------------+
| USER          |
+---------------+
1 row in set (0.01 sec)

mysql> select * from USER;
+----+------------+-----------+------------+-----------+-----------+------------+
| id | first_name | last_name | email      | username  | password  | regdate    |
+----+------------+-----------+------------+-----------+-----------+------------+
|  1 | charan     | charan    | Charan@tej | Charantej | charantej | 2026-04-16 |
+----+------------+-----------+------------+-----------+-----------+------------+
1 row in set (0.00 sec)

mysql>
```

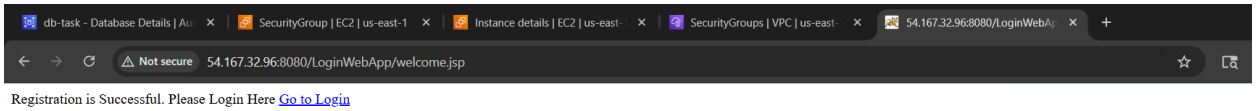
Testing Outcome

Registration Test

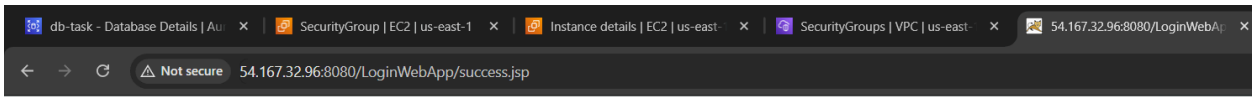
- User form submitted successfully
- Data inserted into `users` table
- Redirected to welcome page

Login Test

- User credentials validated successfully
- Login page functioning as expected



Registration is Successful. Please Login Here Go to Login



Welcome Charantej Log out

Key Learnings

- Launching and configuring EC2
- Creating and configuring RDS MySQL
- Security group communication
- Deploying Java web application on Tomcat
- Connecting application server with RDS
- Verifying data persistence in database

Conclusion

This project successfully demonstrates deployment of a **Java Login and User Registration web application** using:

- **Amazon EC2 as application server**
- **Amazon RDS MySQL as backend database**
- **Tomcat as web server**

The application was successfully tested by verifying that user registration data is stored in the RDS database.